Mock Interview: GFS

Test your knowledge of GFS's architecture by answering these questions.

Quiz ?

Statement

How does GFS scale?

Answer

GFS scales by adding ChunkServers to the cluster. The master gradually moves data from existing ChunkServers to the newly added servers. Since the master stores all metadata in memory (for faster operations), GFS is limited by how much memory the master has. At Google, this is not a serious problem in practice as the master maintains less than 64 bytes of metadata for each 64 MB chunk, but as the system has grown in usage, problems have emerged with the centralized master scheme.Despite the separation of control and data flow and the performance optimization of the master, it is emerging as a bottleneck in the design. Despite the reduced amount of metadata (because of the large chunk size), the amount of metadata stored by the master is increasing to a level where it is getting difficult to keep all the metadata in main memory.

Hide Answer

Quiz ?

Statement

How does GFS ensure fault-tolerance and reliability of data?

Answer

Fault tolerance in GFS is achieved through data replication and maintaining a metadata transactions log for the master. Replication GFS replicates data to multiple nodes for reliability and fault tolerance. By default, each chunk has three replicas stored on three different ChunkServers on possibly three different racks. Operation log For fault tolerance and to handle a master crash, all metadata changes are written to the disk onto an operation log. This operation log is also replicated onto remote machines. Checkpointing Master's state is periodically serialized to disk and then replicated, so that on recovery, a master may load the checkpoint into memory, replay any subsequent operations from the operation log, and be available again very quickly.

Hide Answer

Quiz ?

Statement

How does GFS manage high availability?

Answer

Each chunk is replicated onto multiple ChunkServers to ensure high availability. By default, GFS stores three replicas of each chunk. To ensure maximum data availability, master distributed replicas on different racks, so that clients can still read or write in case of a rack failure. To handle master server crash, GFS stores the checkpoint in a compact B-tree-like form that can be directly mapped into memory and used for namespace lookup without extra parsing. This speeds up the recovery and improves availability.

Hide Answer

Quiz ?

Statement

How does GFS perform master failover?

Answer

GFS relies on an external monitoring infrastructure to detect the master failure and switch the traffic to a backup master server.

Hide Answer

Quiz ?

Statement

What benefits does GFS get from a single master?

Answer

The benefit of having a single, centralized master is that it has a global view of the file-system, and hence, it can make optimum management decisions, for example, related to chunk placement and replication.

Hide Answer

Quiz ?

Statement

What problems does GFS face by having a single master?

Answer

As the number of files grows, it becomes impossible to store all files' metadata in a single master's RAM. Secondly, as the number of clients grows, the single master becomes a performance bottleneck as it does not have enough CPU power to serve them.

Hide Answer

Quiz ?

Statement

How does GFS ensure file data integrity?

Answer

GFS uses checksumming to detect the corruption of stored data. Each chunk is broken down into 64 KB blocks, and each block has a corresponding 32-bit checksum. Like other metadata, checksums are kept in memory and stored persistently with logging, separate from file data. ChunkServers verifies the checksum of data blocks that overlap the read/write range before performing the operation.

Hide Answer

Quiz ?

Statement

How does GFS decouple control flow from data flow, and what benefits does it provide?

Answer

Client communicates with the master for all metadata-related operations like creating or deleting files, looking up files, etc., but all data transfers happen directly between the client and ChunkServers. This decoupling of control flow from data flow minimizes the master's involvement in reads and writes, so that it does not become a bottleneck. Clients never read or write file data through the master. Instead, a client asks the master which ChunkServers it should contact. The client caches this information for a limited time and interacts with the ChunkServers directly for many subsequent operations. Furthermore, while performing a write operation, GFS separates control flow from data flow to ensure data consistency. In a write operation, the data flows from the client to the nearest ChunkServer, which then forwards it to other replicas until all ChunkServers that store replicas for that chunk have received the data. The control (the write request) flow goes from the client to the primary ChunkServer for that chunk. The primary then forwards the request to all the secondaries. This ensures that the primary is in control of the order of writes even if it receives multiple concurrent write requests. All replicas will have data written in the same sequence.

Hide Answer

Quiz ?

Statement

How does lazy space allocation help GFS?

Answer

Each chunk replica is stored as a plain Linux file on a ChunkServer. GFS does not allocate the whole 64MB of disk space when creating a chunk. Instead, as the client appends data, the ChunkServer, lazily extends the chunk. This lazy space allocation avoids wasting space due to internal fragmentation. Internal fragmentation refers to having unused portions of the 64 MB chunk. For example, if we allocate a 64 MB chunk and only fill up 20 MB, the remaining space is unused.

Hide Answer

Quiz ?

Statement

Can GFS handle small files efficiently?

Answer

GFS can store small files efficiently as it lazily extends the chunks based on the data size. This lazy space allocation scheme avoids wasting storage space due to internal fragmentation. Having said that, GFS has a large chunk size (64MB), which can have its disadvantages while reading. Since a small file will have one or a few chunks, the ChunkServers storing those chunks can become hotspots if a lot of clients are accessing the same file. To handle this scenario, GFS stores such files with a higher replication factor and also adds a random delay in the start times of the applications accessing these files.

Hide Answer

Quiz ?

Statement

What are reference counts in context of snapshotting?

Answer

Reference counts are part of the implementation of copy-on-write for snapshots. Snapshots in GFS are initially zero-copy. This means that when GFS creates a snapshot, it does not copy the chunks data. Instead, it increases the reference counter of each chunk. Later, when a client makes a request to write to one of these chunks, the master notices its reference count is greater than one. At this point, the master asks each ChunkServer holding the replica to make a copy of the chunk so that the client can update the copy. Snapshotting with reference counts provide following benefits. 1. It makes creating a snapshot inexpensive, as GFS is not making the copy of the data. 2. It delays the copy until it is absolutely required, hoping that not all chunks will be modified, and hence, GFS can avoid making some copies.

Hide Answer

Quiz ?

Statement

Let's assume C1 is the primary replica ChunkServer of a chunk, and there is a network partition between the master and C1. When the master notices this, it will designate some other ChunkServer as primary, say C2. Since C1 did not actually fail, are there now two primaries for the same chunk?

Answer

If we have two primaries, both might apply different updates to the same chunk – making the chunk inconsistent. To prevent this, GFS uses leases. In the aforementioned scenario, the master grants C1 a 60-second lease to be primary. C1 knows to stop being primary when its lease expires. The master will not grant a lease to C2 until the previous lease to C1 expires. Hence, C2 will not start acting as primary until after C1 stops.

Hide Answer

Quiz ?

Statement

How does GFS handle data consistency?

Answer

Master guarantees data consistency by ensuring the order of mutations on all replicas and using chunk version numbers. If a replica has an incorrect version, it is garbage collected. GFS guarantees at-least-once write for writers. This means that records could be written more than once (although rarely). It is the responsibility of the readers to deal with these duplicate chunks. This is achieved by having checksums and serial numbers in the chunks, which help readers to filter and discard duplicate data. GFS does not guarantee strong consistency. As described above, there can be duplicate data. In other words, GFS preferred performance and simplicity of design over data correctness.

Hide Answer

Quiz ?

Statement

As GFS preferred performance and simplicity over correctness, how did it work for GFS?

Answer

For distributed systems, providing strong data consistency usually results in a complex architecture where different components need to communicate with each other to develop consensus. By relaxing consistency requirements, the design can be simple, providing good performance. For example, GFS optimizes for MapReduce applications, which need high read performance for large files and are fine to deal with duplicate records or inconsistent reads. On the other hand, GFS would not be a good fit for systems requiring consistent data, for example, stockbroker or bank transactions.

Hide Answer

← Previous
Quiz: GFS

Next →
Hadoop Distributed File System: Introduction

☐ Mark as Completed